Comandando a fita RGB IOTBT701 de um notebook Windows: Ambilight via BLE

Relatório analítico, 29 out – 2 nov 2026 (4 sessões) • github.com/Ne-collab228/iotbt701-ambilight • GUI v1.5

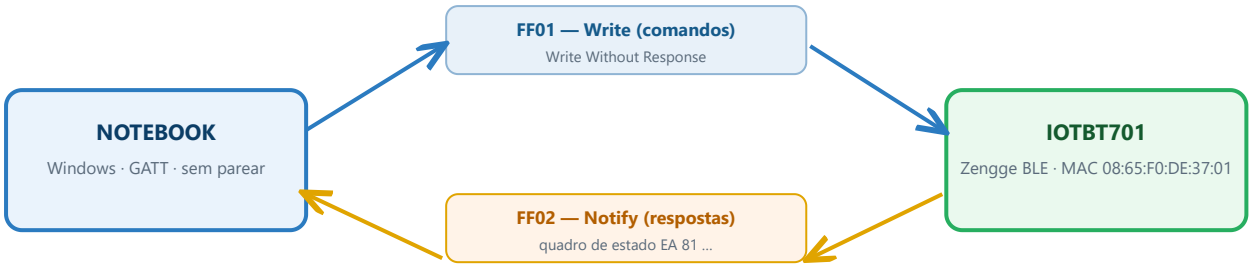
v1.5 GUI + iniciar com Windows + arquivo de registro + janela «LINK LOST»	**10** commits, MIT, repositório público	**5** idiomas da documentação (EN/RU/ES/ZH/PT-BR)	**16.9/s** ritmo sustentado de escritas (soak de 24,4 min)	**3** «falsos limites» eliminados por medição

Objetivo: a fita RGB **IOTBT701** (controlador Zengge/LEDnetWF via BLE, MAC 08:65:F0:DE:37:01) espelha a cor média da tela do notebook. Sem app de celular, sem pareamento, Win32/.NET puro. Resultado: um produto GUI pronto, um repositório público com documentação completa do protocolo e um conjunto de ferramentas CLI — além disso, cada «limite» que a princípio havíamos anotado para nós mesmos foi testado e honestamente revogado.

1. Linha do tempo

Data	Marco	O que aconteceu
29–30 out	**Protocolo decifrado**	Varredura BLE sem parear (PairAsync do Windows sempre devolve «Failed» — normal neste dispositivo, não é erro). GATT direto, sessão com MaintainConnection. Embrulho de transporte decodificado: flags/contador/contadores de fragmento/0x0B delimitador/checksum = sum(payload)&0xFF. Ligar/desligar verificados byte a byte contra as referências da integração lednetwf_ble (Home Assistant).
30 out	**Cor**	Os templates RGB antigos (31/41/42/46) são ignorados: a família IOTBT exige o formato v3 `E0 01 00 A1` + hue/sat/bri. 12 matizes seguidos ✓. Descoberta: graus de matiz ímpares são corrompidos pelo firmware → quantizar para pares (erro de 1°, invisível).
30 out	**Ambilight v1.0**	AmbilightOT.exe: captura de tela, média ponderada de cor, suavização EMA, cenas escuras → branco, reconexão automática.
31 out	**Publicação**	GitHub: docs PROTOCOL/AMBILIGHT/BUILDING, README ×5, GUI com seletor de idioma na própria janela (52ea6c8), v1.1 — correção de branco quente (o «branco» de fábrica puxa para azul), v1.2 — autosave de ajustes + curva de saturação contra o «centro lavado» dos LEDs baratos.
1 nov	**v1.3 — feedback**	A fita finalmente RESPONDEU: canal de consulta (família 0x0A, sem checksum) em FF01 → quadro de estado `EA 81 … F0 hue sat bri` em FF02. lednotify.exe. Limite documentado com honestidade: os registradores da fita só são escritos pelo app do celular. Brilho: a escada 100→130→255 provou «o byte é porcentagem 0..100»; >100 corrompe a cor.
1 nov	**v1.4 — 20 Hz**	Removemos o nosso próprio travão de «≥350 ms»: sustentado de 16,9 cmd/s a 50 ms, fila em rajada de 456/s sem perdas, soak 3600/3600 em 213 s. GUI: o campo «intervalo» controla tudo (amostragem da tela = teto de escrita, mín. 50 ms).
2 nov	**v1.5 + caça ao bug**	Caixa «iniciar com Windows» (fonte de verdade: atalho em shell:startup), ambilight_log.txt com precisão de ms, botão «registro de conexão» e janela automática «LINK LOST» ao desconectar sozinha. Em paralelo o usuário relata travamento da fita a 50 ms após 15–30 min — em investigação (§7).

2. Mapa de serviços BLE da fita



Serviço extra FE00: FF11 (escrita) · FF22 (notify) — canal secundário do mesmo protocolo, desnecessário

A fita tem uma única «porta principal»: comandos saem pela FF01, respostas chegam pela FF02.

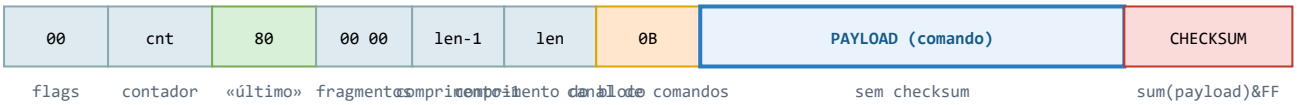
Algoritmo de conexão (início e reconexão)

```
1. BluetoothLEDevice.FromBluetoothAddressAsync(0x0865F0DE3701)   até 6 tentativas × 5 s
2. GattSession.FromDeviceIdAsync(...) → session.MaintainConnection = true
   aguardar ConnectionStatus == Connected                          até 6 tentativas × 2 s
3. GetGattServicesAsync() → FFFF: FF01 (escrever) / FF02 (escutar)
4. Escrita: FF01, WriteWithoutResponse; cada comando pinta na hora
5. Ao perder o enlace → ciclo completo de reconexão a cada ~6 s (infinito)
```

> **Por que sem pareamento:** PairAsync do Windows sempre falha com esta fita — mas pareamento não é preciso: as características GATT estão abertas. Parear apenas bloqueia o acesso.

3. A gramática do protocolo (tudo verificado na fita real)

3.1 Embrulho de transporte: cada byte no seu lugar



Fórmula do checksum decifrada contra as referências lednetwf_ble: soma de TODOS os bytes do payload a partir de 0x0B inclusive, módulo 256. Verificada byte a byte.

3.2 Comando de cor (v3): 23 bytes que pintam a cor

```
00 XX 80 00 00 0F 10 0B | E0 01 00 A1 HH SS BB 00 00 00 00 14 00 00 | CS
                        |   família IOTBT     hue sat% bri%         |  sum & 0xFF
HH = (hue << 7 | sat) >> 8 == hue/2   ·  SS = sat % (0..100)  ·  BB = bri % (0..100)
```

Comando	Payload após 0x0B	Estado
Ligar / desligar	`3B 23/24 00…32 00 00`	✓ verificado
Cor (v3)	`E0 01 00 A1 HH SS BB …`	✓ 12 matizes
Branco	`3B A1 00 00 64 …` (sat=0)	✓
Templates RGB 31/41/42/46	`31 R G B …`	✗ a fita ignora
Grau de matiz ímpar	byte_lo = sat+128	✗ firmware corrompe → quantizar para par

bri > 100	escala percentual	✗ corrompe a cor → teto 100
Consulta de estado	`0B 81 8A 8B 96` (SEM checksum!)	✓ v1.3
Resposta de estado (FF02)	`EA 81 01 00 AD 0A 23 25 6E 37 F0 hue…`	✓ decodificada

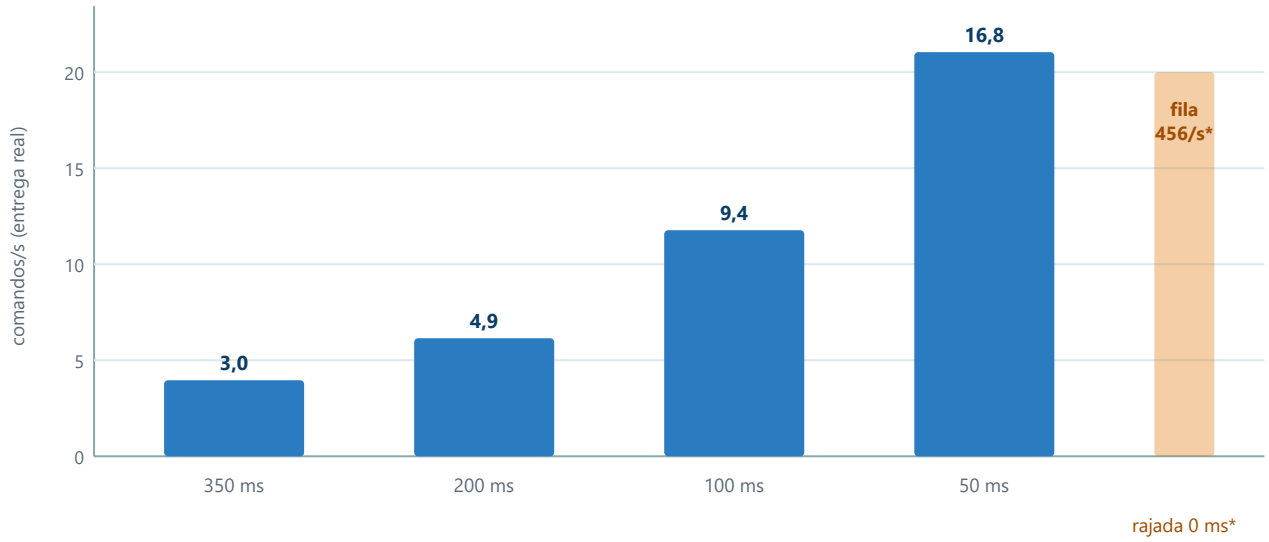
3.3 Por que os templates «31 R G B» falhavam: esta fita é de outra família

Do QUICK_REFERENCE: `product_id = 0x00` (IOTBT/Telink) não é um único tipo de dispositivo — **três famílias de comandos distintas** o compartilham. Nossa fita é a terceira família («embrulho v3»: envelope E0 01 00 A1). Os templates antigos pertencem a outras famílias — uma língua estrangeira para este aparelho.

Família	Liga/Desl.	Cor	Efeitos	Exemplos
Telink	0x71	`E2 0B hue lvl`	E0 02	IOTBT812
Segment	0x3B	`E1 03 …`	E1 01	IOTBT6BA
embrulho v3 (nossa)	0x3B	`E0 01 00 A1 …`	—	ZJ-BBLA (PR #101)

4. Gráficos de desempenho

4.1 Ritmo real de entrega em intervalos distintos



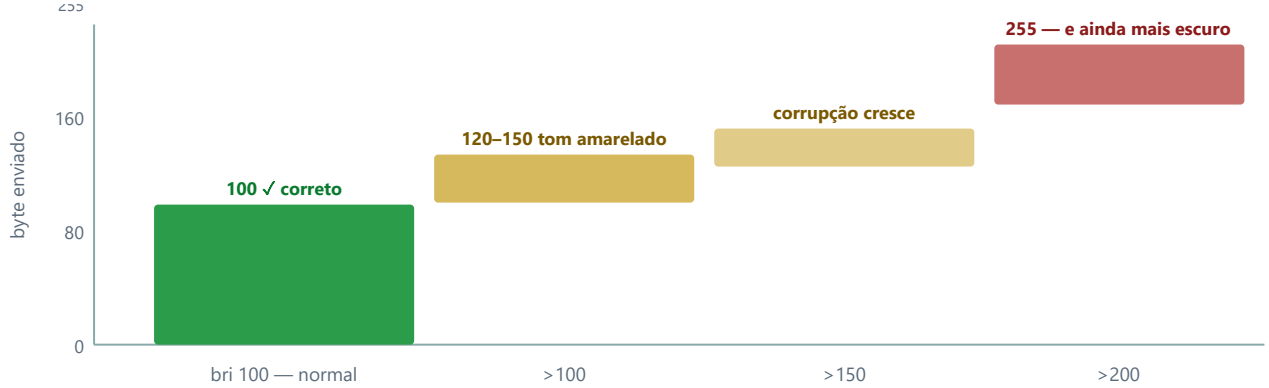
*Rajada = 100 comandos sem pausas: Windows enfileira sem perdas, mas o rádio só transporta ~17/s — esse é o limite físico da conexão. Acima de 17 Hz nada fica mais suave, só mais tarde.

4.2 Testes de resistência (ambas as corridas: 3 min e 24 min a 50 ms)

Corrida	Comandos	Entregues	Falhas	Ritmo	Duração	Vão máx.
1 nov · soak #1	3600	3600/3600	0	16,9/s	213 s	141 ms
2 nov · soak #2	24000	24000/24000	0	16,4/s	24,4 min	125 ms

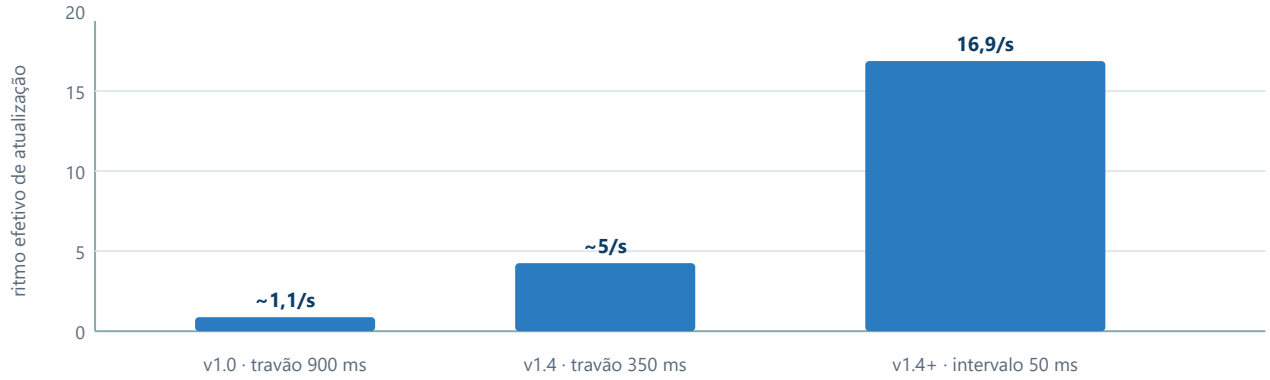
O ritmo nunca se degrada → o firmware não sofre sob carga contínua de 20 Hz. Após cada corrida: consulta de controle + restauração do laranja: sucesso.

4.3 Escada de brilho (o defeito do firmware acima de 100)



O byte bri é uma PORCENTAGEM 0..100. Valores >100 são aceitos mas corrompem a cor (tom amarelo-esverdeado; rumo a 200–255 também escurece) — defeito de firmware, não física dos LEDs.

4.4 Evolução: os travões que nós mesmos impusemos, e como foram retirados



Três «limites» não eram física, mas excesso de cautela: «1–3 cmd/s», «precisa de vão de 350 ms», «brilho é 0–255». Retirados por medição: 20 Hz seguro, teto 100.

5. Ferramentas

Ferramenta	Propósito
AmbilightOT.exe (GUI)	Carro-chefe: ajustes (intervalo 50 ms…3 s, ganhos de sat/bri, brilho mín., limiar de branco, correção de branco quente), 5 idiomas de interface, autosave, reconexão automática; v1.5 acrescenta a caixa «iniciar com Windows» (atalho em shell:startup), arquivo de registro e janela automática «LINK LOST».
ledctl.exe	Console: white/on/off/rgb/hsv/sweep2 — controle manual de cor.
lednotify.exe	Feedback e experimentos: query/listen/ladder/rate/soak/send.
blescan.exe	Varredura BLE de 25 s — verifica «a fita está viva no ar» (quando NINGUÉM segura o enlace).
build_*.ps1	Compilação com csc.exe + fachadas GAC + WinMD — sem Visual Studio.

6. Lições técnicas (o que não está na documentação oficial)

- **Emparelhar é nocivo e desnecessário**: PairAsync do Windows sempre falha nesta família, mas as características GATT estão abertas.
- **O celular é monopolista do enlace**: com o Surplife conectado, a fita fica inacessível do notebook (e vice-versa). Testes só com o BT do celular desligado.
- **Fita conectada NÃO anuncia**: enquanto está em conexão não aparece nas varreduras. «Não está no ar» muitas vezes significa «alguém retém seu enlace», não «morreu». O diagnóstico de morte só faz sentido sem ninguém conectado.
- **PowerShell 5.1 não serve**: eventos WinRT (varredura, notify) não são entregues (status=Started, packets=0); só as escritas funcionam. Toda ferramenta interativa é C#.
- **Modos dinâmicos do controle** (arco-íris/perseguição) atropelam os comandos de cor estática — ponha a fita em modo estático pelo controle antes dos testes.
- **O checksum pode não ser estrito**: uma vez a fita pintou a cor com checksum errado. Ainda assim confiamos na fórmula verificada.
- **O byte de sequência importa**: um contador de pacotes repetido pode ser descartado (dos docs DISCOVERIES) — sempre incrementar.
- **Registradores ≠ nossos comandos**: o quadro de estado mostra registradores que só o celular escreve. Nossos comandos mudam a cor fisicamente enquanto os registradores ficam velhos — o feedback mostra que o enlace vive, não é espelho do que enviamos.

7. Caça ao «travamento» da fita (2 nov, em andamento)

> **Sintoma relatado pelo usuário:** com a GUI a intervalo de 50 ms, após 15–30 minutos a fita para de atualizar; a GUI tenta reconectar sem êxito; «a fita não está no ar». É preciso cortar e religar a energia. A luz NÃO apaga — segue pintando a última cor aplicada.

Descartado: o próprio ritmo de escrita

A reprodução direta falhou: lednotify soak 50 ms × 24,4 min (24 000 comandos, mesmo ritmo da GUI) — **0 falhas**, enlace vivo, ritmo estável de 16,4/s; consulta posterior ok. No dia anterior, outros 3,5 min limpos. Carga constante de 20 Hz não mata o firmware. Detalhe-chave dado pelo usuário: **a parte de LED continua viva, a parte de rádio morre** — os comandos não são recebidos e a fita para de anunciar.

Hipóteses (por probabilidade)

#	Hipótese	Raciocínio	Teste
1	Deadlock de firmware: um travamento raro da pilha BLE desencadeado pela duração / padrões próprios de tráfego da GUI (transições EMA, repetições), não pela taxa bruta	O soak envia um fluxo de comandos extremamente uniforme (varredura completa do arco-íris); a GUI gera um perfil real com muitas repetições de bytes idênticos	Corrida de 1–2 h pela GUI real; o arquivo de registro v1.5 marca o momento
2	Problema da pilha/driver do Windows (enlace zumbi)	«Não está no ar» provavelmente foi conferido pelo notebook cujo enlace envelheceu — a fita podia estar viva; o soak prova que o firmware é resistente	No momento do travamento, abrir Surplife no celular: vê → problema do PC; não vê → firmware
3	Alimentação/térmica: queda de tensão do controlador sob carga prolongada	Cortar e religar resolve — coerente	Reproduzir ao longo de dias, anotar se a descarga ajuda
4	Estado da fita: um modo dinâmico (arco-íris) brigando com comandos estáticos	Queixas parecidas nos issues da integração	Manter a fita em modo estático pelo controle

O kit v1.5 já está pronto para isto: o registro grava carimbos de tempo em ms e o ponto exato do LINK LOST; a janela automática avisa o usuário no instante da queda. Próxima corrida: GUI a 50 ms por 1–2 horas, celular à mão para a checagem ao vivo no instante do travamento.

8. GitHub

README ×5 · docs/PROTOCOL.md §1–§4 · docs/AMBILIGHT.md · docs/BUILDING.md · src/*.cs · LICENÇA MIT · Releases v1.1–v1.4

Release	Tema	Ativos
v1.1	Correção de branco quente	AmbilightOT.exe
v1.2	Autosave + curva de saturação	AmbilightOT.exe
v1.3	Feedback FF02 + brilho 0..100	lednotify.exe, ledctl.exe
v1.4	Modo rápido: 20 Hz verificado	AmbilightOT.exe, lednotify.exe
v1.5 (publicada)	Caixa de inicialização + registro + janela «LINK LOST»	AmbilightOT.exe, lednotify.exe + relatórios PDF ×5

Commits: `e820fcc` início → `16c6009` docs EN → `dc549a0` localizações → `52ea6c8` idiomas da GUI → `7e90b94` branco quente → `6c5a10a` v1.2 → `3b9f305` MIT/capturas → `bf46696` v1.3 → `975371a` v1.4.

9. Backlog

Tarefa	Estado	Notas
Leitura de estado (FF02)	feito (v1.3)	query + decodificação + lednotify; limitação documentada.
Escala de brilho 0..100	feito (v1.3)	teto 100 confirmado.
Ritmo de 20 Hz	feito (v1.4)	soak de 24,4 min limpo; «acima de 17 Hz» não faz sentido.
Início com Windows	feito (v1.5)	caixa na GUI + atalho em shell:startup.
Registro de falhas	feito (v1.5)	arquivo + botão + janela «LINK LOST».
Diagnóstico do travamento	em andamento	corrida GUI 1–2 h; checagem por celular no momento da queda.
Seletor de monitor (lista)	backlog	lista suspensa em vez de «principal / todas». Candidata v1.6.
Interpolação de transições	backlog	2–3 micro-quadros de fade entre cores em vez de «teleportes».
Matiz 1° em vez de 2°	pesquisa	graus ímpares se corrompem — testar primeiro.
Gama de brilho	backlog	meios-tons mais ricos em cenas escuras.

10. Conclusões

- **O protocolo está exaustivamente decodificado** — do embrulho de transporte aos recantos: família v3, quantização a graus pares, brilho em porcentagem, canal de consulta, quadro de estado FF02, registradores. Tudo no PROTOCOL.md.
- **Ambilight é um produto, não um script**: GUI em 5 idiomas, autosave, reconexão automática, inicialização com Windows, registro que se abre sozinho. Compila-se sem Visual Studio.
- **Cada «limite» foi levantado por experimento**: três medos próprios dissipados por medição — 16,9/s sustentado, teto 100, travão de 350 ms desnecessário.
- **A regra «verificar com honestidade» valeu a pena**: o único item aberto é um travamento raro que a carga sintética não reproduz; a caça continua pela GUI real + celular batedor no instante da falha.

Criado em 2 nov 2026 · Todos os números são medições reais dos logs do lednotify (rate/soak/ladder), não teoria · Fonte: Desktop\моды\лента\Project_Report*.html